◆ CREATING THE ERROR RESPONSE

```
return ResponseEntity
        .status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(new EmailResponseDTO("SERVER_ERROR", 0.0));
```

This is the heart of the exception handler.

When an exception occurs, this code creates the response that will be sent back to the client.

◆ WHAT DOES return MEAN HERE?

Remember:

```
public ResponseEntity<EmailResponseDTO>
handleException(Exception e)
```

The return type is:

```
ResponseEntity<EmailResponseDTO>
```

Therefore Java expects:

```
return someResponseEntity;
```

This line fulfills that promise.

◆ WHAT IS ResponseEntity?

Think of a normal HTTP response.

Every HTTP response contains:

```
Status Code
Headers
Body
```

Example:

```
HTTP 200 OK
```

with

```
{
  "result":"SPAM",
  "confidence":1.0
}
```

ResponseEntity allows us to build all parts of that response.

Think of it as a response container.

```
ResponseEntity
├── Status
├── Headers
└── Body
```

◆ WHAT IS .status() ?

```
.status(HttpStatus.INTERNAL_SERVER_ERROR)
```

sets the HTTP status code.

Think:

```
ResponseEntity
↓
Set Status
↓
Set Body
↓
Return Response
```

◆ WHAT IS HttpStatus.INTERNAL_SERVER_ERROR ?

```
HttpStatus.INTERNAL_SERVER_ERROR
```

is a Spring constant.

It represents:

```
500 Internal Server Error
```

These two mean the same thing:

```
HttpStatus.INTERNAL_SERVER_ERROR
```

↓

```
500
```

Spring prefers constants because they are easier to read.

◆ WHY 500?

A 500 error means:

```
Something went wrong
inside the server.
```

Examples:

Flask unavailable

↓

Connection failure

NullPointerException

↓

Application error

Unexpected exception

↓

Server error

All of these are server-side problems.

Therefore:

```
500 Internal Server Error
```

is appropriate.

◆ WHAT DOES .body() DO?

After setting the status code:

```
.status(...)
```

we add the response body:

```
.body(...)
```

Think:

```
Status = 500

+

Body = ?
```

The body is what the client actually receives.

◆ WHAT BODY ARE WE SENDING?

```
new EmailResponseDTO(
    "SERVER_ERROR",
    0.0
)
```

This creates an EmailResponseDTO object.

Exactly like in EmailService.

◆ WHAT OBJECT IS CREATED?

The constructor:

```
public EmailResponseDTO(
    String result,
    double confidence)
```

receives:

```
result = "SERVER_ERROR"
confidence = 0.0
```

Resulting object:

```
EmailResponseDTO
{
    result = "SERVER_ERROR",
    confidence = 0.0
}
```

◆ WHY USE SERVER_ERROR?

Suppose Flask crashes.

Without this handler:

User may receive a long technical error.

Example:

```
Connection refused
```

or

```
NullPointerException
```

Instead we return:

```
SERVER_ERROR
```

A clean, consistent message.

◆ WHY USE 0.0 ?

Remember:

```
EmailResponseDTO
```

requires:

```
result
confidence
```

two values.

Since an error occurred, there is no prediction confidence.

The developer uses:

```
0.0
```

as a placeholder value.

Think:

```
Prediction available?
↓
No
↓
Confidence available?
↓
No
↓
Use 0.0
```

◆ WHAT DOES THE CLIENT RECEIVE?

Object created:

```
new EmailResponseDTO(
    "SERVER_ERROR",
    0.0
)
```

Spring converts it to JSON:

```
{
   "result":"SERVER_ERROR",
   "confidence":0.0
}
```

At the same time, HTTP status becomes:

```
500 Internal Server Error
```

Final response:

```
Status: 500

Body:
{
   "result":"SERVER_ERROR",
   "confidence":0.0
}
```

◆ COMPLETE ERROR FLOW

User sends request

↓

Controller

↓

Service

↓

Exception occurs

↓

Spring finds

```
@ExceptionHandler(Exception.class)
```

↓

Calls

```
handleException(e)
```

↓

Creates

```
ResponseEntity
```

↓

Status = 500

↓

Body = EmailResponseDTO

↓

Spring converts DTO to JSON

↓

Client receives error response

◆ SUMMARY

```
.status(HttpStatus.INTERNAL_SERVER_ERROR)
```

sets the HTTP status code to 500.

```
.body(...)
```

sets the response body.

```
new EmailResponseDTO(
    "SERVER_ERROR",
    0.0
)
```

creates a DTO representing an error.

Spring converts the DTO into JSON and sends it back to the client along with the HTTP 500 status code.

This completes the entire `GlobalExceptionHandler` class.